



Harbinger: Utility Functions & Evaluation Functions



Data Analytics Lab

dal.global@grupo.cefet-rj.br | <https://eic.cefet-rj.br/dal>



Residual Aggregation: L1 and L2 Distance Functions

In time series anomaly detection, **residuals** represent the difference between observed and predicted values, highlighting deviations from expected behavior. Since these residuals are often multi-dimensional (e.g., from a multivariate model), we need to aggregate them into a single scalar signal that can then be fed into a thresholding stage for anomaly identification.

L1 Norm (Manhattan Distance)

Calculated as the sum of the absolute values of the individual residual components. $\sum_{i=1}^n |r_i|$

- **Robust to outliers:** Large isolated spikes have less influence on the aggregated signal, since absolute values grow linearly.
- **Treats all deviations equally:** Each residual component contributes proportionally, regardless of magnitude.
- **Best for:** Datasets with occasional large but non-anomalous fluctuations, or when robustness is preferred over sensitivity.

L2 Norm (Euclidean Distance)

Calculated as the square root of the sum of the squared individual residual components. $\sqrt{\sum_{i=1}^n r_i^2}$

- **Sensitive to large deviations:** Squaring residuals amplifies the contribution of large errors, making significant spikes stand out.
- **More "natural" distance:** Corresponds to Euclidean geometry and is widely used in statistical modeling.
- **Best for:** Scenarios where any large deviation is a strong signal of an anomaly or critical event.

Choosing between L1 and L2 is critical; the resulting scalar signal directly influences the effectiveness of the subsequent anomaly **thresholding** stage. L1 is often preferred when robustness to occasional large, non-anomalous fluctuations is desired, while L2 is chosen when any significant deviation signals a critical event.

Distance Functions: L1 and L2

Harbinger provides utility functions for aggregating residual magnitudes into scalar signals that feed directly into threshold-based event detection. L1 aggregation is more robust to outliers, while L2 amplifies larger deviations — the choice shapes which events ultimately get flagged.

```
# Carregar pacotes e instanciar utilitários
library(daltoolbox)
library(harbinger)

hutils <- harutils()

# Gerar resíduos sintéticos
values <- rnorm(30, mean = 0, sd = 1)

# Agregação L1 da magnitude dos resíduos
v1 <- hutils$har_distance_l1(values)
har_plot(harbinger(), v1)

# Agregação L2 da magnitude dos resíduos
v2 <- hutils$har_distance_l2(values)
har_plot(harbinger(), v2)
```

Step-by-step

- **harutils()** — instantiates the utility object exposing all helper functions
- **rnorm()** — generates synthetic residuals for a controlled demonstration
- **har_distance_l1** — sum of absolute deviations; robust to outliers in the time series
- **har_distance_l2** — sum of squared deviations; sensitive to large residual spikes
- **har_plot()** — visualizes the aggregated signal before thresholding

📌 **Key concept:** L1 and L2 are residual aggregations that feed into the threshold stage of event detection — choosing between them directly influences which change-points and anomalies are surfaced.

Thresholding Strategies for Anomaly Detection

Thresholding is the final step in anomaly detection, converting a continuous scalar residual signal into a binary decision: anomaly or not. It determines the boundary beyond which deviations from expected behavior are considered significant events.

Gaussian 3-Sigma Rule

Flags values beyond $\mu \pm 3\sigma$, assuming residuals follow a normal distribution. This is a common statistical approach for identifying outliers based on standard deviation.

- **Pros:** Simple, fast, and widely understood.
- **Cons:** Highly sensitive to non-normal distributions in residuals; can misclassify anomalies if data is skewed.

Boxplot / IQR Method

Defines anomaly bounds using the interquartile range (IQR). Values are flagged if they fall outside $Q3 + 1.5 \times IQR$ or $Q1 - 1.5 \times IQR$.

- **Pros:** Robust to skewed distributions and outliers present within the residual signal itself.
- **Cons:** May be less sensitive to extreme but valid deviations if the distribution is very broad.

Ratio-Based Thresholding

Flags values that deviate by a relative proportion (e.g., a multiple) from a dynamic baseline (like a moving average). This adapts the threshold to changing signal scales.

- **Pros:** Highly effective when the scale or variance of residuals changes significantly over time.
- **Cons:** Requires careful tuning of the ratio parameter; can be less precise for stable residual patterns.

Harbinger allows for full modularity, enabling seamless interchangeability of these strategies via `model$har_outliers`. No single strategy is universally best; the optimal choice depends entirely on the specific distribution shape and characteristics of your time series residuals.

Thresholding Rules & Detection Grouping

With residuals in hand, the next step is deciding what counts as an anomaly. This slide compares three threshold strategies — Gaussian 3-sigma, Boxplot/IQR, and Ratio — and shows how to swap them into the same ARIMA-based event detection pipeline without changing any other component.

```
data(examples_anomalies)
dataset <- examples_anomalies$simple
hutils <- harutils()

# Baseline: ARIMA + L2 + Gaussian 3-sigma
model <- hanr_arima()
model <- fit(model, dataset$serie)
detection <- detect(model, dataset$serie)
har_plot(model, attr(detection,"res"),
  detection, dataset$event,
  yline = attr(detection,"threshold"))

# Swap in Boxplot/IQR threshold
model <- hanr_arima()
model$har_outliers <- hutils$har_outliers_boxplot
model <- fit(model, dataset$serie)
detection <- detect(model, dataset$serie)
har_plot(model, attr(detection,"res"),
  detection, dataset$event,
  yline = attr(detection,"threshold"))
```

```
# Swap in Ratio threshold
model <- hanr_arima()
model$har_outliers <- hutils$har_outliers_ratio
model <- fit(model, dataset$serie)
detection <- detect(model, dataset$serie)
har_plot(model, attr(detection,"res"),
  detection, dataset$event,
  yline = attr(detection,"threshold"))
```

Step-by-step

- **hanr_arima()** — ARIMA-based detector for time series event detection
- **har_outliers_boxplot** — threshold via IQR (Tukey, 1977), robust to skewed residuals
- **har_outliers_ratio** — threshold via relative deviation from the mean
- **attr(detection, "res")** — residuals extracted from the fitted model
- **ylines** — renders the threshold boundary on the plot for visual inspection

📌 **Key concept:** Replacing `model$har_outliers` swaps the threshold rule without touching the rest of the pipeline — full modularity across all event detection workflows.

Detection Grouping: From Runs to Events

When a threshold is crossed in time series anomaly detection, it often remains crossed for multiple consecutive time steps. This produces a "run" of detections for what is, in essence, a single underlying event, leading to inflated false positive counts and noisy output.

Grouping strategies address this by treating a contiguous block of flagged time steps as a single event. This significantly improves the clarity and actionability of anomaly detection results.

Default: No Grouping

Every flagged time step is reported as a separate detection. This can obscure the true timing of events and lead to an overcount of anomalies, making it difficult to pinpoint the most significant deviation.

Peak selection is critical for downstream evaluation and alerting, as it assigns a precise, single timestamp to each event. This mechanism is controlled via `model$har_outliers_check`, offering modularity akin to other pipeline components.

Grouped: Peak Selection

Using `har_outliers_checks_highgroup`, within each contiguous run, only the time step with the highest residual magnitude (the "peak") is retained as the sole detection point. This provides a precise timestamp for each anomaly.

Combining Distance, Threshold, and Grouping

Building on the previous threshold strategies, these advanced combinations pair L1 distance with different threshold rules — and introduce grouping to retain only the highest-magnitude index within contiguous detection runs.

```
# L1 + Gaussian (default)
model <- hanr_arima()
model$har_distance <- hutils$har_distance_l1
model <- fit(model, dataset$serie)
detection <- detect(model, dataset$serie)
har_plot(model, attr(detection,"res"),
  detection, dataset$event,
  yline = attr(detection,"threshold"))

# L1 + Boxplot/IQR
model <- hanr_arima()
model$har_distance <- hutils$har_distance_l1
model$har_outliers <- hutils$har_outliers_boxplot
model <- fit(model, dataset$serie)
detection <- detect(model, dataset$serie)
har_plot(model, attr(detection,"res"),
  detection, dataset$event,
  yline = attr(detection,"threshold"))
```

```
# L1 + Boxplot + keep only the peak of each group
model <- hanr_arima()
model$har_distance <- hutils$har_distance_l1
model$har_outliers <- hutils$har_outliers_boxplot
model$har_outliers_check <-
  hutils$har_outliers_checks_highgroup
model <- fit(model, dataset$serie)
detection <- detect(model, dataset$serie)
har_plot(model, attr(detection,"res"),
  detection, dataset$event,
  yline = attr(detection,"threshold"))
```

Step-by-Step

- **har_distance_l1** — swaps the model's distance function to L1
- **har_outliers_boxplot** — IQR-based threshold combined with L1 distance
- **har_outliers_checks_highgroup** — within contiguous runs, retains only the highest-magnitude change-point
- Each model slot is independently replaceable

📌 **Key concept:** Grouping via `highgroup` prevents multiple event detections for the same anomaly — reducing false positives across contiguous segments of the time series.

Strict Evaluation: Confusion Matrix Metrics

Rigorous evaluation is crucial for any event detection system. Without it, a detector that flags everything could appear effective, but would be useless in practice. The confusion matrix provides a foundational framework to assess performance objectively in time series anomaly detection.

True Positive (TP)

A detection that coincides exactly with a labeled event.

False Positive (FP)

A detection at a time step where no labeled event exists.

False Negative (FN)

A labeled event that the detector failed to identify.

True Negative (TN)

No event and no detection, indicating correct inaction.

Accuracy

$$(TP + TN) / (TP + FP + FN + TN)$$

Precision

$TP / (TP + FP)$ – how many detections are real events.

Recall

$TP / (TP + FN)$ – how many real events were caught.

F1 Score

Harmonic mean of precision and recall, balancing both metrics.

A **strict criterion** means a detection is considered a True Positive only if it aligns **exactly** (same time index) with a labeled event, offering no tolerance for temporal shifts. This approach is most appropriate for clean, well-labeled datasets where event timestamps are precise and meaningful.

Strict Event Detection Evaluation with `har_eval()`

`har_eval()` computes accuracy, precision, recall, and F1 using a strict criterion: a detection is a true positive only if it aligns exactly with a labeled event.

```
# Load data and inspect structure
library(daltoolbox)
library(harbinger)

data(examples_anomalies)
dataset <- examples_anomalies$simple
head(dataset)
## serie event
## 1.000 FALSE
## 0.969 FALSE

# Visualize raw time series
har_plot(harbinger(), dataset$serie)

# Configure MLP-based detector
model <- hanr_ml(
  ts_mlp(ts_norm_gminmax(),
    input_size = 5,
    size = 3,
    decay = 0))
```

```
# Fit model and run detection
model <- fit(model, dataset$serie)
detection <- detect(model, dataset$serie)

# Inspect detected indices
print(detection |> dplyr::filter(event == TRUE))
## idx event type
## 50 TRUE anomaly

# Evaluate with hard metrics
evaluation <- evaluate(har_eval(),
  detection$event,
  dataset$event)
print(evaluation$confMatrix)
## detection TRUE FALSE
## TRUE 1 0
## FALSE 0 100
```

Step-by-Step Breakdown

- **`head(dataset)`** — inspects `serie` and `event` columns
- **`hanr_ml(ts_mlp(...))`** — MLP-based regression detector
- **`ts_norm_gminmax()`** — global min-max normalization
- **`evaluate(har_eval(), ...)`** — computes full confusion matrix
- Result: 1 TP, 0 FP, 0 FN — perfect detection

📌 **Key concept:** `har_eval()` enforces strict evaluation — detections must align exactly with labeled events to count as true positives.

Soft Evaluation: Tolerance-Aware Metrics (SoftED)

Strict evaluation often fails in real-world scenarios where event exact timestamps are uncertain due to sensor delays, approximate labels, or gradual onsets. Penalizing detections that are slightly off as full false positives can be too harsh and misrepresent actual detector performance.

Strict Evaluation: Limitations

- Assumes perfect event timing.
- No tolerance for temporal shifts.
- Detection slightly off is a full False Positive.
- Doesn't reflect real-world imprecision.

SoftED: The Tolerance-Aware Solution

- Defines a sliding window (`sw_size`) around each event.
- Detections within the window receive partial credit.
- Produces "soft" TP, FP, FN for nuanced assessment.
- Better reflects performance with temporal uncertainty.

By transforming standard binary counts into "soft" versions, SoftED allows for a more forgiving and realistic assessment. The resulting soft confusion matrix seamlessly integrates into the familiar precision, recall, and F1 score framework, providing a robust evaluation whenever temporal uncertainty exists.

Reference: Salles, R. et al. (2024). SoftED: Metrics for soft evaluation of time series event detection. Computers and Industrial Engineering. doi:10.1016/j.cie.2024.110728

Soft Evaluation with `har_eval_soft()` (SoftED)

SoftED provides tolerance-aware evaluation by matching event detections against ground truth within sliding time windows and assigning partial credit. This approach is especially useful when there is temporal uncertainty around the true onset of an event.

```
# Same setup as the previous slide
data(examples_anomalies)
dataset <- examples_anomalies$simple

model <- hanr_ml(
  ts_mlp(ts_norm_gminmax(),
    input_size = 5,
    size = 3,
    decay = 0))
model <- fit(model, dataset$serie)
detection <- detect(model, dataset$serie)

# Hard evaluation (baseline reference)
evaluation <- evaluate(model,
  detection$event, dataset$event,
  evaluation = har_eval())
print(evaluation$confMatrix)
```

```
# Soft evaluation with tolerance window
sw_size = 5
result <- evaluate(model,
  detection$event, dataset$event,
  evaluation = har_eval_soft(sw_size = 5))
print(result$confMatrix)

# Can also be called without the model object
result <- evaluate(
  har_eval_soft(sw_size = 5),
  detection$event,
  dataset$event)
print(result$confMatrix)
## detection TRUE FALSE
##   TRUE  1   0
##  FALSE  0 100
```

Step-by-step

- **`har_eval_soft(sw_size = 5)`** — applies a tolerance window of ± 5 time points
- Detections falling within the window receive partial credit
- Can be invoked with or without the `model` object
- In this example, hard and soft evaluation yield identical results, since event detection is exact

📌 **Key concept:** SoftED is preferable when temporal uncertainty is present — minor misalignments in event detection should not be penalized as outright errors. Reference: Salles et al. (2024), *Computers and Industrial Engineering*.

Harbinger Core Concepts: A Recap

Harbinger provides a robust and flexible framework for time series event detection, emphasizing modularity and comprehensive evaluation.

Distance Functions

Choose between **L1** (robust to outliers) and **L2** (amplifies large deviations) for residual aggregation.

Flexible Thresholding

Interchangeable strategies: Gaussian 3-sigma, Boxplot/IQR, or Ratio, easily swappable via `model$har_outliers`.

Grouping Anomalies

`har_outliers_checks_highgroup` identifies peak detections within contiguous anomaly runs, reducing false positives.

Strict Evaluation

`har_eval()` uses confusion matrix metrics with **exact event matching** for rigorous assessment.

Soft Evaluation (SoftED)

`har_eval_soft()` offers tolerance-aware evaluation using sliding windows, ideal for temporal uncertainty.

Additional Sources

Core Textbook

Ogasawara, E., Salles, R., Porto, F., Pacitti, E. *Event Detection in Time Series*. Springer, 2025. doi:10.1007/978-3-031-75941-3

SoftED

Salles, R. et al. (2024). SoftED: Metrics for soft evaluation of time series event detection. *Computers and Industrial Engineering*. doi:10.1016/j.cie.2024.110728

Anomaly Detection Survey

Chandola, V., Banerjee, A., Kumar, V. (2009). Anomaly detection: A survey. *ACM Computing Surveys*, 41(3), 1–58.

Fine-Tuning Detection Criteria

Sobrinho, E. P., Souza, J., Lima, J., Giusti, L., Bezerra, E., Coutinho, R., Baroni, L., Pacitti, E., Porto, F., Belloze, K., e Ogasawara, E. "Fine-Tuning Detection Criteria for Enhancing Anomaly Detection in Time Series." *Simpósio Brasileiro de Banco de Dados (SBBD)*, SBC, set. 2025, p. 209–221. doi:10.5753/sbbd.2025.247063

Harbinger on GitHub

Source code, examples, issue tracker, and documentation available at: github.com/cefet-rj-dal/harbinger